Day 2 — Advanced Prompt Engineering

> 🎯 **Goal**: Build mastery over prompt structures and behaviors.

> ⏱️ **Time**: 1.5–2h theory + 2–3h practice

> 📦 **Deliverable**: A reusable advanced prompt library with validators

---

Table of Contents

1. Why Prompt Engineering Matters
2. Zero-Shot Prompting
3. Few-Shot Prompting
4. Chain-of-Thought (CoT)
5. Structured Output Prompting
6. Function Calling
7. Prompt Failure Modes
8. Advanced Techniques
9. Practice Exercises

Why Prompt Engineering Matters

The same model can give wildly different results depending on how you prompt it. Prompt engineering is **the** skill that separates someone who "uses ChatGPT" from someone who **builds with LLMs**.

```
Same model, same question, different prompts:

Prompt 1 (bad): "Is this code secure?"
Response: "Yes, it looks fine." ← Useless

Prompt 2 (good): "You are a senior security engineer. Analyze the following
code for OWASP Top 10 vulnerabilities. For each vulnerability found, provide:
1. The vulnerability type (e.g., SQL Injection)
2. The exact line(s) affected
3. Why it's vulnerable
4. A fixed version of the code
```

```
If no vulnerabilities are found, explain what security measures are in place."
Response: Detailed, actionable security analysis ← Valuable
```

Zero-Shot Prompting

Zero-shot means giving the model a task **without any examples**. You rely entirely on the model's pre-trained knowledge.

When to Use

- Simple, well-defined tasks
- Tasks the model has seen plenty of during training
- When you need speed/simplicity

Anatomy of a Good Zero-Shot Prompt

```
┌─────────────────────────────────────────────────────┐
│                   ZERO-SHOT PROMPT                  │
│                                                     │
│  1. ROLE/PERSONA                                    │
│     "You are a senior backend engineer..."          │
│                                                     │
│  2. TASK DESCRIPTION                                │
│     "Analyze the following API endpoint for..."     │
│                                                     │
│  3. INPUT DATA                                      │
│     "```python\ndef get_user(id): ...\n```"         │
│                                                     │
│  4. OUTPUT FORMAT SPECIFICATION                     │
│     "Respond with a JSON object containing..."      │
│                                                     │
│  5. CONSTRAINTS                                     │
│     "Do not include explanations outside the JSON." │
│                                                     │
└─────────────────────────────────────────────────────┘
```

Examples

```
# Classification (zero-shot)
SENTIMENT_PROMPT = """You are a sentiment analysis system.

Classify the following text into exactly one category:
- POSITIVE
- NEGATIVE
- NEUTRAL
```

```
Text: "{text}"

Respond with only the category name, nothing else."""

# Extraction (zero-shot)
ENTITY_PROMPT = """Extract all named entities from the following text.

Text: "{text}"

Respond as a JSON array of objects with these fields:
- "entity": the extracted text
- "type": one of PERSON, ORGANIZATION, LOCATION, DATE, MONEY

Example output format:
[{{"entity": "John", "type": "PERSON"}}]"""

# Transformation (zero-shot)
SQL_PROMPT = """You are a SQL expert. Convert the following natural language
query into a PostgreSQL query.

Database schema:
- users(id, name, email, created_at)
- orders(id, user_id, amount, status, created_at)
- products(id, name, price, category)
- order_items(order_id, product_id, quantity)

Natural language query: "{query}"

Respond with only the SQL query, no explanations."""
```

Few-Shot Prompting

Few-shot means providing **examples** of the desired input→output behavior. The model learns the pattern from your examples and applies it to new inputs.

Why Few-Shot Works

```
The model performs "in-context learning" — it doesn't update its weights,
but it recognizes the pattern in your examples and continues it.

Think of it as:
  "I showed you 3 examples of how I want this done.
   Now do the same thing for this new input."
```

How to Choose Good Examples

```
Rules for selecting few-shot examples:

1. DIVERSE: Cover different cases, not just the easy ones
   Bad:  3 examples of positive sentiment
   Good: 1 positive, 1 negative, 1 neutral

2. REPRESENTATIVE: Match the distribution of real inputs
   Bad:  All short sentences when real data has long paragraphs
   Good: Mix of lengths matching your actual data

3. EDGE CASES: Include at least one tricky example
   Bad:  All straightforward cases
   Good: Include an ambiguous case showing the desired behavior

4. CONSISTENT FORMAT: Output format must be identical across all examples
   Bad:  Example 1 uses JSON, Example 2 uses plain text
   Good: All examples use the exact same JSON structure

5. ORDERED: Put the hardest/most relevant example last (recency bias)
```

Examples

```
  CLASSIFICATION_FEW_SHOT = """Classify customer support tickets into categories.

  Examples:

  Ticket: "I can't log into my account, it says password incorrect"
  Category: AUTHENTICATION

  Ticket: "My payment was charged twice for order #12345"
  Category: BILLING

  Ticket: "The app crashes when I try to upload a photo larger than 10MB"
  Category: BUG_REPORT

  Ticket: "Can you add dark mode to the mobile app?"
  Category: FEATURE_REQUEST

  Ticket: "I want to cancel my subscription but I can't find the option"
  Category: ACCOUNT_MANAGEMENT

  Now classify this ticket:
  Ticket: "{ticket_text}"
  Category:"""

  CODE_REVIEW_FEW_SHOT = """Review code and provide feedback in JSON format.

  Example 1:
  Code: `user_input = request.args.get('query'); db.execute(f"SELECT * FROM users WHER
  Review: {{"severity": "CRITICAL", "issue": "SQL Injection", "fix": "Use parameterize

  Example 2:
```

```
  Code: `password = 'admin123'`
  Review: {{"severity": "HIGH", "issue": "Hardcoded credential", "fix": "Use environme

  Example 3:
  Code: `items = [x for x in range(1000000)]`
  Review: {{"severity": "MEDIUM", "issue": "Memory inefficiency", "fix": "Use generato

  Now review this code:
  Code: `{code}`
  Review:"""
```

How Many Examples?

```
┌──────────┬──────────────────────────────────────────────┐
│ Examples │ When to Use                                  │
├──────────┼──────────────────────────────────────────────┤
│ 0        │ Simple tasks, well-defined output format     │
│ 1-2      │ Format clarification, style matching         │
│ 3-5      │ Complex classification, nuanced behavior     │
│ 5-10     │ Very specific patterns, custom DSLs          │
│ 10+      │ Rarely needed; consider fine-tuning instead  │
└──────────┴──────────────────────────────────────────────┘

Warning: More examples = more tokens = higher cost + slower response
  Each example might use 50-200 tokens
  5 examples × 100 tokens = 500 tokens of context consumed
```

Chain-of-Thought (CoT)

Chain-of-Thought prompting asks the model to **show its reasoning steps** before giving a final answer. This dramatically improves performance on complex tasks.

Why CoT Works

```
Without CoT:
  Q: "Roger has 5 tennis balls. He buys 2 more cans of 3.
      How many tennis balls does he have?"
  A: "11" ← The model jumps to the answer (sometimes wrong)

With CoT:
  Q: "Roger has 5 tennis balls. He buys 2 more cans of 3.
      How many tennis balls does he have? Let's think step by step."
  A: "Roger starts with 5 balls.
      He buys 2 cans, each with 3 balls.
      2 cans × 3 balls = 6 new balls.
      5 + 6 = 11 total balls.
      The answer is 11." ← The model reasons through it
```

```
Why it helps:
  1. Forces sequential reasoning (not jumping to conclusions)
  2. Each step provides context for the next step
  3. Errors are visible and correctable
  4. The model effectively "shows its work"
```

Types of CoT

```
1. ZERO-SHOT CoT: Just add "Let's think step by step"
   Simple but effective. Works for most reasoning tasks.

2. FEW-SHOT CoT: Provide examples with reasoning chains
   More reliable, controls the reasoning format.

3. SELF-CONSISTENCY CoT: Generate multiple reasoning paths,
   take the majority answer.
   Prompt → Generate 5 different CoTs → Vote on final answer
   More expensive but more accurate.

4. TREE-OF-THOUGHT: Explore multiple reasoning branches,
   evaluate each, backtrack if needed.
   Best for complex planning/puzzle tasks.
```

Implementation

```
# Zero-shot CoT
REASONING_PROMPT = """Solve the following problem. Think step by step,
showing your reasoning clearly before giving the final answer.

Problem: {problem}

Solution:
Let me work through this step by step.

Step 1:"""

# Few-shot CoT
MATH_COT_PROMPT = """Solve math word problems by reasoning step by step.

Problem: A store sells notebooks for $3 each. If you buy 5 or more,
you get a 20% discount. How much do 7 notebooks cost?

Solution:
Step 1: Calculate the base price. 7 notebooks × $3 = $21
Step 2: Check if discount applies. 7 ≥ 5, so yes, 20% discount.
Step 3: Calculate discount. 20% of $21 = $4.20
Step 4: Subtract discount. $21 - $4.20 = $16.80
Answer: $16.80
```

```
Problem: A train travels at 60 mph for 2 hours, then at 80 mph for
1.5 hours. What is the total distance?

Solution:
Step 1: Distance at 60 mph. 60 × 2 = 120 miles
Step 2: Distance at 80 mph. 80 × 1.5 = 120 miles
Step 3: Total distance. 120 + 120 = 240 miles
Answer: 240 miles

Problem: {problem}

Solution:"""

# Self-consistency (multiple samples, take majority)
import openai
from collections import Counter

def solve_with_self_consistency(problem, n_samples=5):
    """Generate multiple reasoning paths and vote on the answer."""
    answers = []

    for _ in range(n_samples):
        response = openai.chat.completions.create(
            model="gpt-4",
            messages=[{
                "role": "user",
                "content": REASONING_PROMPT.format(problem=problem)
            }],
            temperature=0.7,  # Some randomness for diverse paths
            max_tokens=500
        )

        text = response.choices[0].message.content
        # Extract final answer (last line or after "Answer:")
        answer = extract_answer(text)
        answers.append(answer)

    # Majority vote
    vote = Counter(answers).most_common(1)[0]
    return vote[0], vote[1] / n_samples  # answer, confidence

def extract_answer(text):
    """Extract the final answer from a CoT response."""
    lines = text.strip().split("\n")
    for line in reversed(lines):
        if "answer" in line.lower() or "=" in line:
            # Extract number or final value
            import re
            numbers = re.findall(r'[\d,]+\.?\d*', line)
            if numbers:
                return numbers[-1]
    return lines[-1].strip()
```

Structured Output Prompting

For production systems, you need the model to output **valid, parseable data** — not free-form text.

JSON Output

```
# Basic JSON enforcement
JSON_PROMPT = """Analyze the following customer review and extract structured data.

Review: "{review}"

Respond with ONLY a valid JSON object (no markdown, no explanation) matching
this exact schema:
{{
  "sentiment": "positive" | "negative" | "neutral",
  "rating_estimate": 1-5,
  "topics": ["string"],
  "key_phrases": ["string"],
  "requires_response": true | false,
  "urgency": "low" | "medium" | "high"
}}"""

# Using system message for format enforcement
SYSTEM_MSG = """You are a JSON-only API. You NEVER output anything except
valid JSON. No markdown code blocks, no explanations, no preamble.
Your entire response must be parseable by JSON.parse()."""
```

Schema-Enforced Output

```
# Provide the exact JSON Schema for validation
SCHEMA_PROMPT = """Extract entities from the text and return as JSON.

JSON Schema (your output MUST conform to this):
{{
  "type": "object",
  "required": ["entities", "summary"],
  "properties": {{
    "entities": {{
      "type": "array",
      "items": {{
        "type": "object",
        "required": ["name", "type", "confidence"],
        "properties": {{
          "name": {{"type": "string"}},
          "type": {{"type": "string", "enum": ["PERSON", "ORG", "LOCATION"]}},
          "confidence": {{"type": "number", "minimum": 0, "maximum": 1}}
        }}
```

```
      }}
    }},
    "summary": {{"type": "string", "maxLength": 200}}
  }}
}}

Text: "{text}"

JSON output:"""
```

DSL (Domain-Specific Language) Output

```
# Sometimes JSON is overkill — define a simple DSL
TASK_DSL_PROMPT = """Convert the user request into a task specification
using this format:

ACTION: <verb>
TARGET: <what to act on>
PARAMS:
  - key: value
  - key: value
CONDITIONS:
  - condition description
OUTPUT: <expected result type>

User request: "{request}"

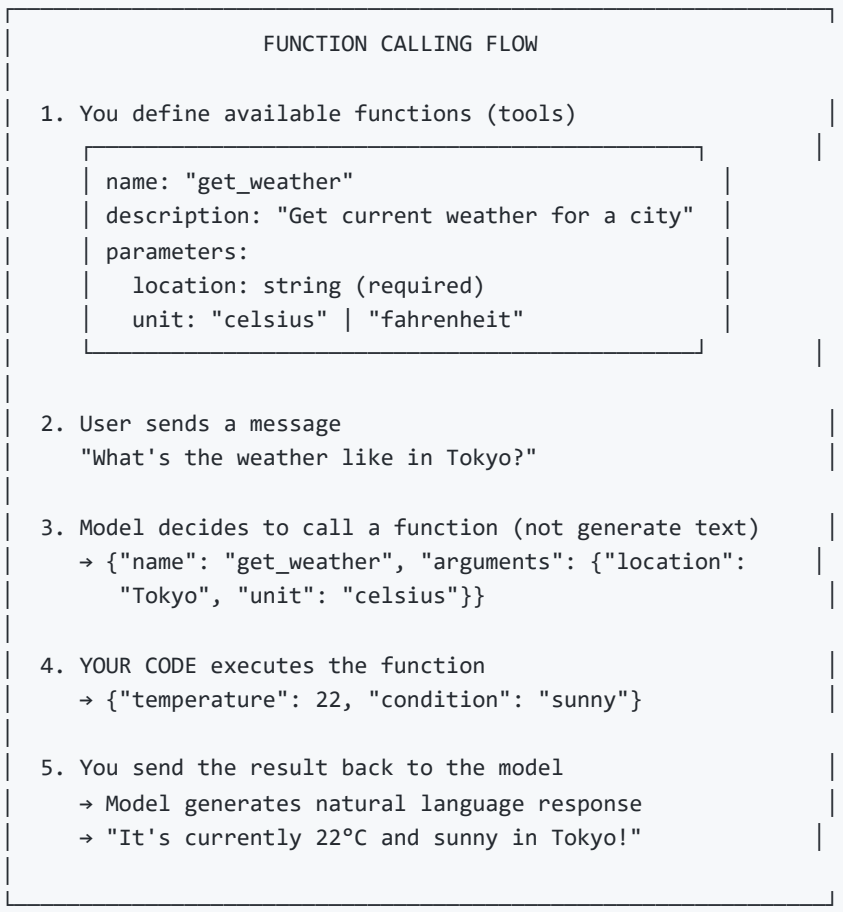
Task specification:"""

# Example output:
# ACTION: SEARCH
# TARGET: database.users
# PARAMS:
#   - field: email
#   - pattern: *@company.com
#   - limit: 100
# CONDITIONS:
#   - created_at > 2024-01-01
#   - status = active
# OUTPUT: list of user records
```

Function Calling

Function calling lets the model decide **when and how to call external tools**. This is the foundation of tool-using agents (Day 3).

How Function Calling Works



Implementation with OpenAI

```python
import openai
import json

# Step 1: Define your tools
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get the current weather in a given location",
            "parameters": {
                "type": "object",
                "properties": {
                    "location": {
                        "type": "string",
                        "description": "City name, e.g., 'San Francisco, CA'"
                    },
                    "unit": {
                        "type": "string",
                        "enum": ["celsius", "fahrenheit"],
```

```
                        "description": "Temperature unit"
                    }
                },
                "required": ["location"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "search_database",
            "description": "Search the product database",
            "parameters": {
                "type": "object",
                "properties": {
                    "query": {"type": "string", "description": "Search query"},
                    "category": {
                        "type": "string",
                        "enum": ["electronics", "clothing", "food"],
                        "description": "Product category filter"
                    },
                    "max_results": {
                        "type": "integer",
                        "description": "Maximum results to return",
                        "default": 5
                    }
                },
                "required": ["query"]
            }
        }
    }
]

# Step 2: Your actual function implementations
def get_weather(location, unit="celsius"):
    """Simulate a weather API call."""
    # In production, call a real weather API
    return {"temperature": 22, "condition": "sunny", "location": location}

def search_database(query, category=None, max_results=5):
    """Simulate a database search."""
    return [{"name": f"Product matching '{query}'", "price": 29.99}]

# Map function names to implementations
FUNCTION_MAP = {
    "get_weather": get_weather,
    "search_database": search_database,
}

# Step 3: The conversation loop
def chat_with_tools(user_message):
    messages = [{"role": "user", "content": user_message}]

    # First call — model may decide to use a tool
    response = openai.chat.completions.create(
        model="gpt-4",
```

```
            messages=messages,
            tools=tools,
            tool_choice="auto"  # Let model decide
        )

        msg = response.choices[0].message

        # Check if model wants to call a function
        if msg.tool_calls:
            messages.append(msg)  # Add assistant's tool call message

            for tool_call in msg.tool_calls:
                fn_name = tool_call.function.name
                fn_args = json.loads(tool_call.function.arguments)

                print(f"  → Calling {fn_name}({fn_args})")

                # Execute the function
                result = FUNCTION_MAP[fn_name](**fn_args)

                # Add function result to messages
                messages.append({
                    "role": "tool",
                    "tool_call_id": tool_call.id,
                    "content": json.dumps(result)
                })

            # Second call — model generates final response using function results
            response = openai.chat.completions.create(
                model="gpt-4",
                messages=messages,
                tools=tools
            )
            return response.choices[0].message.content

        # No tool call — direct response
        return msg.content

    # Test it
    print(chat_with_tools("What's the weather in Tokyo and Paris?"))
    # → Calling get_weather({"location": "Tokyo"})
    # → Calling get_weather({"location": "Paris"})
    # → "In Tokyo it's 22°C and sunny. In Paris it's..."
```

Parallel Function Calls

Modern models can request **multiple function calls in a single response**:

```
    # Model's response might contain multiple tool_calls:
    # tool_calls = [
    #     {"id": "call_1", "function": {"name": "get_weather", "arguments": '{"location"
    #     {"id": "call_2", "function": {"name": "get_weather", "arguments": '{"location"
```

```
  # ]
  #
  # You execute BOTH, then send BOTH results back
  # This is more efficient than sequential calls
```

Prompt Failure Modes

Understanding how prompts fail is critical for building reliable systems.

Failure Mode 1: Hallucinations

```
What it is:
  The model generates plausible-sounding but factually incorrect information.

Example:
  Q: "What paper introduced the RLHF technique?"
  A: "RLHF was introduced in 'Training Language Models to Follow
      Instructions with Human Feedback' by Ouyang et al. (2022),
      published at NeurIPS 2022."

  Reality: The paper exists and is correct, BUT sometimes models
  fabricate paper titles, author names, or publication venues.

Mitigation strategies:
  1. Ground responses in provided context (RAG — Day 5)
  2. Ask model to cite specific sources from given documents
  3. Add "If you don't know, say 'I don't know'" to prompt
  4. Use lower temperature (0.0-0.3) for factual tasks
  5. Verify with a second model call or external check
```

Failure Mode 2: Invalid JSON

```
What it is:
  The model outputs text that looks like JSON but isn't valid.

Common issues:
  - Trailing commas: {"a": 1, "b": 2,}  ← invalid
  - Single quotes: {'key': 'value'}  ← invalid JSON
  - Unquoted keys: {key: "value"}  ← invalid
  - Comments: {"key": "value" /* comment */}  ← invalid
  - Markdown wrapping: ```json\n{...}\n```  ← extra text
  - Truncation: {"key": "very long value that gets cu  ← cut off

Mitigation (Validate → Retry → Fix pattern):
```

```python
import json
import re

def extract_json(text):
    """Extract JSON from model output, handling common issues."""
    # Strip markdown code blocks
    text = re.sub(r'^```(?:json)?\s*\n?', '', text.strip())
    text = re.sub(r'\n?```\s*$', '', text.strip())

    # Try parsing directly
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass

    # Try finding JSON object in text
    match = re.search(r'\{.*\}', text, re.DOTALL)
    if match:
        try:
            return json.loads(match.group())
        except json.JSONDecodeError:
            pass

    # Try finding JSON array
    match = re.search(r'\[.*\]', text, re.DOTALL)
    if match:
        try:
            return json.loads(match.group())
        except json.JSONDecodeError:
            pass

    return None

def get_valid_json(prompt, model="gpt-4", max_retries=3):
    """Repeatedly prompt until valid JSON is obtained."""
    import openai

    messages = [
        {"role": "system", "content": "Respond with valid JSON only. "
         "No markdown, no explanations, no text outside the JSON."},
        {"role": "user", "content": prompt}
    ]

    for attempt in range(max_retries):
        response = openai.chat.completions.create(
            model=model,
            messages=messages,
            temperature=0.0 if attempt > 0 else 0.3  # Lower temp on retry
        )

        text = response.choices[0].message.content
        result = extract_json(text)

        if result is not None:
```

```
            return result

        # Add error context for retry
        messages.append({"role": "assistant", "content": text})
        messages.append({
            "role": "user",
            "content": f"That was not valid JSON. Error: {get_json_error(text)}. "
                       f"Please respond with ONLY valid JSON, no other text."
        })
        print(f"  Retry {attempt + 1}/{max_retries}: Invalid JSON, retrying...")

    raise ValueError(f"Failed to get valid JSON after {max_retries} attempts")

def get_json_error(text):
    """Get a helpful error message for invalid JSON."""
    try:
        json.loads(text)
        return "Unknown"
    except json.JSONDecodeError as e:
        return str(e)
```

Failure Mode 3: Mode Collapse / Refusal

```
What it is:
  The model refuses to answer or gives overly generic responses.

Examples:
  - "I can't help with that" (overly cautious alignment)
  - "As an AI language model, I..." (meta-responses)
  - Repeating the question back instead of answering

Mitigation:
  1. Be specific about what you want — vague prompts get vague answers
  2. Use a system prompt that establishes expertise
  3. Frame tasks professionally (not as circumventing safety)
  4. Break complex requests into smaller, clearer steps
```

Failure Mode 4: Instruction Following Failures

```
What it is:
  The model ignores part of your instructions.

Common cases:
  - Includes explanations when you said "JSON only"
  - Uses wrong format despite explicit schema
  - Adds fields not in the schema
  - Ignores length constraints

Why it happens:
  - Prompt is too long (instructions get "lost")
```

```
  - Conflicting instructions
  - Model's training biases override your instructions
  - Important instructions buried in the middle (lost-in-the-middle)

Mitigation:
  1. Put the MOST important instructions at the START and END
  2. Use clear delimiters (###, ---) between sections
  3. Repeat critical constraints
  4. Use structured prompts (numbered lists)
  5. Post-process outputs to enforce format

Position matters (lost-in-the-middle effect):
  ┌──────────────────────────────────────────┐
  │ High recall                              │
  │ ███████                          ███████ │
  │ ███████                          ███████ │
  │ ███████      Low recall          ███████ │
  │ ███████     ▒▒▒▒▒▒▒▒▒▒           ███████ │
  │ ███████     ▒▒▒▒▒▒▒▒▒▒           ███████ │
  │ ███████     ▒▒▒▒▒▒▒▒▒▒           ███████ │
  │                                          │
  │ Beginning    Middle                End   │
  │ of prompt    of prompt           of prompt │
  └──────────────────────────────────────────┘

  Put critical instructions at the beginning and end!
```

Advanced Techniques

System Messages

```
# System messages set the behavior for the entire conversation
# They have the strongest influence on model behavior

SYSTEM_MESSAGES = {
    "json_api": (
        "You are a JSON-only API. Every response must be valid JSON. "
        "Never include markdown, explanations, or any text outside JSON. "
        "If you cannot complete the request, respond with: "
        '{"error": "description of the problem"}'
    ),

    "code_reviewer": (
        "You are a senior software engineer performing code reviews. "
        "Focus only on bugs, security issues, and performance problems. "
        "Do not comment on style or formatting. "
        "Rate each issue as CRITICAL, HIGH, MEDIUM, or LOW severity."
    ),

    "sql_expert": (
```

```
            "You are a PostgreSQL expert. Generate only SQL queries. "
            "Always use parameterized queries (never string interpolation). "
            "Include helpful comments for complex queries. "
            "Consider query performance and suggest indexes when relevant."
        )
    }
```

Prompt Chaining

Break complex tasks into sequential prompts, where each prompt
uses the output of the previous one:



Why chain? Each step is simpler and more reliable than one
mega-prompt trying to do everything at once.

```
import openai

def chain_prompts(text):
    """Multi-step analysis using prompt chaining."""

    # Step 1: Extract topics
    topics_response = openai.chat.completions.create(
        model="gpt-4",
        messages=[{
            "role": "user",
            "content": f"List the 3 main topics in this text as a JSON array "
                       f"of strings. Only output the JSON array.\n\n{text}"
        }],
        temperature=0.0
    )
    topics = json.loads(topics_response.choices[0].message.content)

    # Step 2: Analyze each topic
```

```
    analyses = {}
    for topic in topics:
        analysis_response = openai.chat.completions.create(
            model="gpt-4",
            messages=[{
                "role": "user",
                "content": f"From this text, extract key points about '{topic}'. "
                           f"Return as JSON: {{\"points\": [\"...\"], "
                           f"\"sentiment\": \"positive|negative|neutral\"}}\n\n{text
            }],
            temperature=0.0
        )
        analyses[topic] = json.loads(analysis_response.choices[0].message.content)

    # Step 3: Synthesize
    synthesis_response = openai.chat.completions.create(
        model="gpt-4",
        messages=[{
            "role": "user",
            "content": f"Given this analysis:\n{json.dumps(analyses, indent=2)}\n\n"
                       f"Write a 3-paragraph executive summary."
        }],
        temperature=0.3
    )

    return synthesis_response.choices[0].message.content
```

Meta-Prompting

```
# Use an LLM to generate/optimize prompts for you

META_PROMPT = """You are a prompt engineering expert. Given a task description,
generate an optimized prompt that will produce the best results from GPT-4.

Your prompt should include:
1. A clear system role
2. Precise task description
3. Output format specification
4. Edge case handling
5. At least 2 few-shot examples

Task: {task_description}

Generate the optimized prompt:"""

# Example: Generate a prompt for code documentation
task = "Generate API documentation from Python function signatures"
optimized_prompt = call_llm(META_PROMPT.format(task_description=task))
# → Returns a carefully crafted prompt with examples and format specs
```

Practice Exercises

Exercise 1: Build a Prompt Library

```
"""
Reusable prompt library with template rendering and validation.
"""
import json
import re
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class PromptTemplate:
    """A reusable prompt template with variable substitution."""
    name: str
    system_message: str
    user_template: str
    output_format: str  # "json", "text", "dsv"
    json_schema: Optional[dict] = None
    examples: list = field(default_factory=list)
    temperature: float = 0.3
    max_tokens: int = 1000

    def render(self, **kwargs) -> tuple:
        """Render the prompt with variables. Returns (system, user) messages."""
        user_content = self.user_template.format(**kwargs)

        # Add examples if few-shot
        if self.examples:
            examples_text = "\n\nExamples:\n"
            for i, ex in enumerate(self.examples, 1):
                examples_text += f"\nExample {i}:\n"
                examples_text += f"Input: {ex['input']}\n"
                examples_text += f"Output: {ex['output']}\n"
            user_content = examples_text + "\nNow process this:\n" + user_content

        return self.system_message, user_content

    def validate_output(self, output: str) -> tuple:
        """Validate and parse the model's output. Returns (parsed, errors)."""
        if self.output_format == "json":
            return self._validate_json(output)
        elif self.output_format == "text":
            return output.strip(), []
        return output, []

    def _validate_json(self, output: str) -> tuple:
        """Parse and optionally validate JSON against schema."""
        # Clean markdown
        cleaned = re.sub(r'^```(?:json)?\s*\n?', '', output.strip())
        cleaned = re.sub(r'\n?```\s*$', '', cleaned.strip())
```

```
        try:
            parsed = json.loads(cleaned)
        except json.JSONDecodeError as e:
            return None, [f"Invalid JSON: {e}"]

        errors = []
        if self.json_schema:
            errors = self._check_schema(parsed, self.json_schema)

        return parsed, errors

    def _check_schema(self, data, schema, path="root") -> list:
        """Simple JSON schema validation."""
        errors = []

        if schema.get("type") == "object" and isinstance(data, dict):
            for req in schema.get("required", []):
                if req not in data:
                    errors.append(f"Missing required field '{req}' at {path}")

            props = schema.get("properties", {})
            for key, val_schema in props.items():
                if key in data:
                    errors.extend(
                        self._check_schema(data[key], val_schema, f"{path}.{key}")
                    )

        elif schema.get("type") == "array" and isinstance(data, list):
            item_schema = schema.get("items", {})
            for i, item in enumerate(data):
                errors.extend(
                    self._check_schema(item, item_schema, f"{path}[{i}]")
                )

        elif schema.get("type") == "string" and not isinstance(data, str):
            errors.append(f"Expected string at {path}, got {type(data).__name__}")

        elif schema.get("type") == "number" and not isinstance(data, (int, float)):
            errors.append(f"Expected number at {path}, got {type(data).__name__}")

        if "enum" in schema and data not in schema["enum"]:
            errors.append(f"Value '{data}' at {path} not in {schema['enum']}")

        return errors


# ── PROMPT LIBRARY ──────────────────────────────────────────

PROMPTS = {}

# Sentiment Analysis
PROMPTS["sentiment"] = PromptTemplate(
    name="sentiment",
    system_message="You are a sentiment analysis API. Output valid JSON only.",
    user_template='Analyze the sentiment of this text:\n\n"{text}"',
    output_format="json",
```

```
    json_schema={
        "type": "object",
        "required": ["sentiment", "confidence", "reasoning"],
        "properties": {
            "sentiment": {"type": "string", "enum": ["positive", "negative", "neutra
            "confidence": {"type": "number"},
            "reasoning": {"type": "string"}
        }
    },
    examples=[
        {
            "input": "This product is amazing, best purchase ever!",
            "output": '{"sentiment": "positive", "confidence": 0.95, '
                      '"reasoning": "Strong positive language: amazing, best ever"}'
        },
        {
            "input": "The delivery was late and the item was broken.",
            "output": '{"sentiment": "negative", "confidence": 0.9, '
                      '"reasoning": "Multiple negative events: late delivery, broken
        }
    ],
    temperature=0.0
)

# Code Review
PROMPTS["code_review"] = PromptTemplate(
    name="code_review",
    system_message=(
        "You are a senior engineer performing code review. "
        "Focus on bugs, security issues, and performance. "
        "Output valid JSON only."
    ),
    user_template='Review this code:\n\n```\n{code}\n```',
    output_format="json",
    json_schema={
        "type": "object",
        "required": ["issues", "overall_quality"],
        "properties": {
            "issues": {
                "type": "array",
                "items": {
                    "type": "object",
                    "required": ["severity", "description", "fix"],
                    "properties": {
                        "severity": {"type": "string",
                                     "enum": ["CRITICAL", "HIGH", "MEDIUM", "LOW"]},
                        "description": {"type": "string"},
                        "line": {"type": "number"},
                        "fix": {"type": "string"}
                    }
                }
            },
            "overall_quality": {"type": "string",
                                "enum": ["excellent", "good", "needs_work", "poor"]}
        }
    },
```

```
    temperature=0.0
)

# SQL Generator
PROMPTS["sql_generator"] = PromptTemplate(
    name="sql_generator",
    system_message=(
        "You are a PostgreSQL expert. Generate only the SQL query. "
        "Use parameterized queries with $1, $2, etc. for user inputs. "
        "No explanations."
    ),
    user_template=(
        "Database schema:\n{schema}\n\n"
        "Generate a query for: {question}"
    ),
    output_format="text",
    examples=[
        {
            "input": "Find users who signed up this month",
            "output": "SELECT * FROM users WHERE created_at >= date_trunc('month', C
        }
    ],
    temperature=0.0
)


# ── USAGE EXAMPLE ──────────────────────────────────────────

def demo():
    """Demonstrate the prompt library."""
    # Render a prompt
    prompt = PROMPTS["sentiment"]
    system_msg, user_msg = prompt.render(
        text="The new update is terrible, it broke everything!"
    )
    print(f"System: {system_msg}")
    print(f"User: {user_msg}")

    # Simulate model output and validate
    model_output = '{"sentiment": "negative", "confidence": 0.92, "reasoning": "Stro
    parsed, errors = prompt.validate_output(model_output)
    print(f"\nParsed: {parsed}")
    print(f"Errors: {errors}")  # Should be empty

    # Test with bad output
    bad_output = '{"sentiment": "angry", "confidence": "high"}'
    parsed, errors = prompt.validate_output(bad_output)
    print(f"\nBad output errors: {errors}")
    # → ["Value 'angry' at root.sentiment not in ['positive', 'negative', 'neutral']
    #    "Expected number at root.confidence, got str"]

if __name__ == "__main__":
    demo()
```

Exercise 2: JSON Output with Validate → Retry → Fix

```
"""
Robust JSON extraction with automatic retry and self-correction.
"""
import json
import re
import time
from typing import Optional

class RobustJSONExtractor:
    """Extract valid JSON from LLM outputs with retry logic."""

    def __init__(self, model="gpt-4", max_retries=3):
        self.model = model
        self.max_retries = max_retries
        self.stats = {"attempts": 0, "successes": 0, "retries": 0}

    def extract(self, prompt: str, json_schema: Optional[dict] = None) -> dict:
        """
        Send prompt to LLM and extract valid JSON with retries.

        Flow:
        ┌─────────────────────────────────────────┐
        │  Send prompt to LLM                     │
        │         │                               │
        │         ▼                               │
        │  Parse response as JSON                 │
        │         │                               │
        │    ┌────┴────┐                          │
        │    │ Valid?  │                          │
        │    └────┬────┘                          │
        │   Yes   │  No                           │
        │    │    │                               │
        │    │    ▼                               │
        │    │  Validate against schema           │
        │    │    │                               │
        │    │  ┌─┴──┐                            │
        │    │  │Pass?│                           │
        │    │  └─┬──┘                            │
        │    │ Yes│  No                           │
        │    │    │   │                           │
        │    │    │   ▼                           │
        │    │    │  Send error + retry prompt    │
        │    │    │   │                           │
        │    │    │   └──── (up to max_retries) ─┐ │
        │    │    │                              │ │
        │    ▼    ▼                              │ │
        │  Return parsed JSON                    │ │
        │                                        └─┘ │
        └─────────────────────────────────────────┘
        """
        import openai
```

```
        self.stats["attempts"] += 1

        messages = [
            {
                "role": "system",
                "content": (
                    "You are a JSON-only API. Your ENTIRE response must be "
                    "a single valid JSON object. No markdown code fences, "
                    "no explanations, no text before or after the JSON."
                )
            },
            {"role": "user", "content": prompt}
        ]

        for attempt in range(self.max_retries + 1):
            response = openai.chat.completions.create(
                model=self.model,
                messages=messages,
                temperature=max(0.0, 0.3 - attempt * 0.1),  # Lower temp each retry
                max_tokens=2000
            )

            raw_output = response.choices[0].message.content

            # Try to parse JSON
            parsed = self._try_parse(raw_output)

            if parsed is None:
                error_msg = self._get_parse_error(raw_output)
                if attempt < self.max_retries:
                    self.stats["retries"] += 1
                    messages.append({"role": "assistant", "content": raw_output})
                    messages.append({
                        "role": "user",
                        "content": (
                            f"ERROR: Your response was not valid JSON.\n"
                            f"Parse error: {error_msg}\n\n"
                            f"Please respond with ONLY a valid JSON object. "
                            f"Do not wrap in markdown. Do not add explanations."
                        )
                    })
                    continue
                raise ValueError(f"Failed to parse JSON after {self.max_retries} ret

            # Validate against schema
            if json_schema:
                schema_errors = self._validate_schema(parsed, json_schema)
                if schema_errors:
                    if attempt < self.max_retries:
                        self.stats["retries"] += 1
                        messages.append({"role": "assistant", "content": raw_output}
                        messages.append({
                            "role": "user",
                            "content": (
                                f"The JSON is valid but doesn't match the required s
                                f"Schema errors:\n"
```

```
                                + "\n".join(f"  - {e}" for e in schema_errors)
                                + f"\n\nPlease fix these issues and respond with cor
                            )
                        })
                        continue
                    raise ValueError(f"Schema validation failed: {schema_errors}")

            self.stats["successes"] += 1
            return parsed

    def _try_parse(self, text: str) -> Optional[dict]:
        """Attempt to parse JSON from text, with cleanup."""
        # Direct parse
        text = text.strip()
        try:
            return json.loads(text)
        except json.JSONDecodeError:
            pass

        # Remove markdown fences
        cleaned = re.sub(r'^```(?:json)?\s*\n?', '', text)
        cleaned = re.sub(r'\n?```\s*$', '', cleaned).strip()
        try:
            return json.loads(cleaned)
        except json.JSONDecodeError:
            pass

        # Find JSON object
        match = re.search(r'\{[\s\S]*\}', text)
        if match:
            try:
                return json.loads(match.group())
            except json.JSONDecodeError:
                pass

        return None

    def _get_parse_error(self, text: str) -> str:
        """Get a helpful error message."""
        try:
            json.loads(text)
            return "Unknown error"
        except json.JSONDecodeError as e:
            return f"Line {e.lineno}, Col {e.colno}: {e.msg}"

    def _validate_schema(self, data, schema, path="$") -> list:
        """Validate data against a JSON schema (simplified)."""
        errors = []
        expected_type = schema.get("type")

        type_map = {
            "object": dict, "array": list, "string": str,
            "number": (int, float), "integer": int, "boolean": bool
        }

        if expected_type and expected_type in type_map:
```

```
            if not isinstance(data, type_map[expected_type]):
                errors.append(
                    f"{path}: expected {expected_type}, got {type(data).__name__}"
                )
                return errors

        if expected_type == "object" and isinstance(data, dict):
            for req in schema.get("required", []):
                if req not in data:
                    errors.append(f"{path}: missing required field '{req}'")
            for key, prop_schema in schema.get("properties", {}).items():
                if key in data:
                    errors.extend(self._validate_schema(data[key], prop_schema, f"{p

        if expected_type == "array" and isinstance(data, list):
            for i, item in enumerate(data):
                if "items" in schema:
                    errors.extend(self._validate_schema(item, schema["items"], f"{pa

        if "enum" in schema and data not in schema["enum"]:
            errors.append(f"{path}: '{data}' not in allowed values {schema['enum']}")

        return errors


# ── USAGE ──────────────────────────────────────────────────────

def demo_extractor():
    """Demonstrate robust JSON extraction."""
    extractor = RobustJSONExtractor(max_retries=3)

    schema = {
        "type": "object",
        "required": ["summary", "key_points", "sentiment"],
        "properties": {
            "summary": {"type": "string"},
            "key_points": {
                "type": "array",
                "items": {"type": "string"}
            },
            "sentiment": {
                "type": "string",
                "enum": ["positive", "negative", "neutral", "mixed"]
            }
        }
    }

    result = extractor.extract(
        prompt="Analyze this review: 'Great product but shipping was slow'",
        json_schema=schema
    )

    print(f"Result: {json.dumps(result, indent=2)}")
    print(f"Stats: {extractor.stats}")
```

```
if __name__ == "__main__":
    demo_extractor()
```

✓ Day 2 Deliverable Checklist

- [ ] Built a `PromptTemplate` class with variable substitution
- [ ] Created at least 3 reusable prompts (sentiment, code review, SQL)
- [ ] Implemented JSON validation against schemas
- [ ] Built a Validate → Retry → Fix mechanism
- [ ] Can explain: What is chain-of-thought and why does it help?
- [ ] Can explain: What are the main prompt failure modes?
- [ ] Can explain: Why does instruction position matter?
- [ ] Can explain: When to use few-shot vs zero-shot?

Next → Day 3: Single-Agent Systems (ReAct)